

```

1 #=====
2 # MIT 6.046J 代码精读 • PS8 - 精确算术 Simplex
3 # 源文件: simplex.py (来自 GitHub mit-6046)
4 #=====
5
6 """PS8 Problem 8-1: a real Simplex solver (Bland's rule) for LPs in standard
7 form  $\max c^T x$  s.t.  $Ax \leq b$ ,  $x \geq 0$  (with  $b \geq 0$  so the slack basis is
8 feasible).
9
10 Also solves the specific pset LP and reports the pivots, and computes the dual
11 optimum. Verified against scipy.optimize.linprog and against brute-force vertex
12 enumeration on the feasible polytope.
13 """
14 from __future__ import annotations
15
16 from fractions import Fraction
17 from typing import List, Optional, Tuple
18
19 Number = Fraction
20
21
22 class Simplex:
23     """Dictionary/tableau simplex with Bland's rule (guarantees termination)."""
24
25     def __init__(self, A: List[List[Number]], b: List[Number], c: List[Number]):
26         self.m = len(A) # constraints
27         self.n = len(c) # decision variables
28         self.pivots: List[Tuple[int, int]] = []
29         # tableau columns: n decision vars + m slack vars + RHS
30         self.N = self.n + self.m
31         # basis[i] = variable index basic in row i (slack i initially)
32         self.basis = [self.n + i for i in range(self.m)]
33         # T is (m+1) x (N+1); last row is objective (we store -c and maximize)
34         self.T = [[Fraction(0)] * (self.N + 1) for _ in range(self.m + 1)]
35         for i in range(self.m):
36             for j in range(self.n):
37                 self.T[i][j] = Fraction(A[i][j])
38             self.T[i][self.n + i] = Fraction(1) # slack
39             self.T[i][self.N] = Fraction(b[i])
40         for j in range(self.n):
41             self.T[self.m][j] = Fraction(-c[j]) # objective row (reduced costs)
42
43     def _pivot(self, prow: int, pcol: int) -> None:
44         self.pivots.append((prow, pcol))
45         piv = self.T[prow][pcol]
46         self.T[prow] = [x / piv for x in self.T[prow]]
47         for i in range(self.m + 1):
48             if i != prow and self.T[i][pcol] != 0:
49                 factor = self.T[i][pcol]
50                 self.T[i] = [a - factor * b for a, b in zip(self.T[i], self.T[prow])]
51         self.basis[prow] = pcol
52
53     def solve(self) -> Tuple[str, Optional[Number], Optional[List[Number]]]:
54         """Return (status, objective_value, x*). Assumes b>=0 (Phase-1-free)."""
55         while True:
56             # Bland: entering = smallest index with negative reduced cost
57             pcol = -1
58             for j in range(self.N):
59                 if self.T[self.m][j] < 0:
60                     pcol = j
61                     break
62             if pcol == -1:
63                 break # optimal
64             # ratio test (Bland: smallest basis index on ties)
65             prow = -1
66             best_ratio = None
67             for i in range(self.m):
68                 if self.T[i][pcol] > 0:
69                     ratio = self.T[i][self.N] / self.T[i][pcol]
70                     if (best_ratio is None or ratio < best_ratio or
71                         (ratio == best_ratio and self.basis[i] < self.basis[prow])):
72                         best_ratio = ratio
73                         prow = i
74             if prow == -1:
75                 return ("unbounded", None, None)
76             self._pivot(prow, pcol)
77
78         x = [Fraction(0)] * self.n
79         for i in range(self.m):
80             if self.basis[i] < self.n:
81                 x[self.basis[i]] = self.T[i][self.N]
82         obj = self.T[self.m][self.N] # equals  $c^T x$ 
83         return ("optimal", obj, x)
84
85     def dual_solution(self) -> List[Number]:

```

```

86         """At optimality, dual variable y_i = reduced cost of slack i in the
87         objective row."""
88         return [self.T[self.m][self.n + i] for i in range(self.m)]
89
90
91 def brute_vertices(A, b, c):
92     """Enumerate all polytope vertices (intersections of n tight constraints
93     among the m+n inequalities incl. x>=0) and take the best feasible one.
94     Exact via Fractions. For low dimension only -- an oracle."""
95     from itertools import combinations
96     import fractions
97     m = len(A)
98     n = len(c)
99     # inequalities: A x <= b and -x_j <= 0
100    rows = [ (list(map(Fraction, A[i])), Fraction(b[i]))) for i in range(m) ]
101    for j in range(n):
102        e = [Fraction(0)] * n
103        e[j] = Fraction(-1)
104        rows.append((e, Fraction(0)))
105    best = None
106    best_x = None
107    for combo in combinations(range(len(rows)), n):
108        M = [rows[i][0] for i in combo]
109        rhs = [rows[i][1] for i in combo]
110        x = _solve_linear(M, rhs)
111        if x is None:
112            continue
113        # feasibility
114        ok = all(sum(A[i][j] * x[j] for j in range(n)) <= b[i] + Fraction(0) for i in range(m))
115        ok = ok and all(x[j] >= 0 for j in range(n))
116        if ok:
117            val = sum(c[j] * x[j] for j in range(n))
118            if best is None or val > best:
119                best = val
120                best_x = x
121    return best, best_x
122
123
124 def _solve_linear(M, rhs):
125     """Solve square system M x = rhs exactly; None if singular."""
126     n = len(M)
127     A = [row[:] + [rhs[i]] for i, row in enumerate(M)]
128     for col in range(n):
129         piv = None
130         for r in range(col, n):
131             if A[r][col] != 0:
132                 piv = r
133                 break
134         if piv is None:
135             return None
136         A[col], A[piv] = A[piv], A[col]
137         pv = A[col][col]
138         A[col] = [a / pv for a in A[col]]
139         for r in range(n):
140             if r != col and A[r][col] != 0:
141                 f = A[r][col]
142                 A[r] = [a - f * b for a, b in zip(A[r], A[col])]
143     return [A[i][n] for i in range(n)]
144

```